UNIVERSITY METROPOLITAN TIRANA
FACULTY OF COMPUTER SCIENCE AND IT
Advanced Software Engineering

M.I.A
Metropolitan Information Agent, AI-Powered University Management System

Development Team: Xhoi Ikonomi, Ajna Nushi, Aida Bedaj, Parashqevi Klimi, Loric Hagard
Supervisor : Professor: Igli Hakrama

June 2026

Abstract

M.I.A (Metropolitan Information Agent) is a full-stack, AI-powered university management system developed for Metropolitan University of Tirana. The system consolidates fragmented academic operations into a unified digital platform serving three user roles, such as students, professors, and administrators, through role-specific portals built on a modern React and FastAPI technology stack, backed by a managed Supabase PostgreSQL database and Supabase Storage for file assets.

The central innovation of M.I.A is a conversational AI adviser embedded in the student portal, powered by the Groq API running the LLaMA 3.1 model. This adviser provides real-time, personalized academic guidance in natural language, answering questions about grades, course recommendations, graduation progress, and financial obligations, replacing the current model in which students wait five to seven days for human adviser appointments. The adviser builds its responses from a rich student academic context compiled at request time from six data sources: enrolled courses, grades, attendance records, GPA calculations, chat history, and stored student preferences.

This report provides comprehensive documentation of the system from requirements analysis through design, architecture, implementation, and behavioral modeling. It includes functional requirements, use case analysis, Data Flow Diagrams at Level 0 and Level 1, UML Class Diagrams covering both the academic and financial domains, a Component Diagram, Layered Architecture analysis, Deployment Diagram, Activity Diagrams for key workflows, State Diagrams for assignment and payment lifecycles, and BPMN process flow diagrams for authentication, assignment submission, and the MIA conversational flow. The architecture follows a five-layer design with deliberate trade-offs documented, including the intentional co-location of business logic and data access in a flat router pattern suited to the project's scale and timeline.

The system targets over 5,000 students and 300 faculty members at Metropolitan University of Tirana, with success criteria including a 50 percent reduction in registrar manual processing time, 90 percent professor adoption of the grade recording interface, and a sub-200 millisecond AI response latency. A five-person team delivered the project in a three-month timeline from April to June 2026.

Keywords: university management system, AI academic adviser, FastAPI, React, Supabase, LLM integration, Groq API, role-based access control, conversational AI, software engineering

Table of Contents

1. Introduction

1.1 Project Overview

M.I.A (Metropolitan Information Agent) represents a comprehensive, full-stack, AI-powered university management system developed for Metropolitan University of Tirana. The system consolidates fragmented academic operations into a unified platform serving three distinct user roles: Students, Professors, and Administrators.

The core vision behind M.I.A is straightforward yet transformative: eliminate friction in university operations by providing students with instant AI-powered academic guidance, giving professors one unified workspace for all course-related operations, and providing administrators complete visibility and control over institutional data.

The standout innovation is a conversational AI adviser embedded directly in the student portal, powered by Groq or Gemini APIs. This adviser provides personalized academic guidance in natural language, addressing a critical bottleneck where students currently wait 5-7 days for adviser appointments that could be resolved in seconds. Students can ask questions like ”When will I graduate?” or ”What courses should I take next semester?” and receive immediate, contextual recommendations based on their academic profile.

The project timeline spans three months from April to June 2026, with a five-person development team structured around two frontend developers, two backend developers, and one AI specialist. The target user base includes over 5,000 students and 300+ faculty members at Metropolitan University of Tirana.

This report documents the complete design, architecture, and execution strategy for the M.I.A system. It gives an in-depth look at how the system addresses current institutional challenges, the technical approach taken to solve these problems, the team structure managing the project, and the criteria by which success will be measured.

The scope of this report extends from high-level vision and problem analysis through detailed system architecture, feature specifications for all three user roles, technical implementation details, project timeline and milestones, risk management strategies, and success criteria with measurable evaluation metrics.

This document is intended for multiple audiences: Metropolitan University of Tirana stakeholders and decision-makers who need to understand what the system will do and why; the development team and technical staff who need to understand how it will be built; project investors and sponsors who need to understand the scope and timeline; course instructors and academic advisers who may contribute feedback; and future maintainers and contributors who will need to understand the system design.

1.2 Motivation and Vision

Metropolitan University of Tirana currently operates academic management through a collection of disconnected systems, manual processes, and paper-based workflows that create significant friction for all stakeholders. The core vision behind M.I.A is straightforward yet transformative: eliminate friction in university operations by providing students with instant AI-powered academic guidance, giving professors one unified workspace for all course-related operations, and providing administrators complete visibility and control over institutional data.

The standout innovation is a conversational AI adviser embedded directly in the student portal, powered by the Groq API. This adviser provides personalized academic guidance in natural language, addressing a critical bottleneck where students currently wait 5–7 days for adviser appointments that could be resolved in seconds. Students can ask questions such as “When will I graduate?” or “What courses should I take next semester?” and receive immediate, contextual recommendations based on their academic profile.

1.3 Client Requirements and Context

Metropolitan University of Tirana is a leading private higher education institution in Albania serving approximately 5,000 students across five schools and 20+ academic departments. The university has motivated students, committed faculty, and modern campus facilities. However, academic operations rely on fragmented systems and manual processes that create inefficiency and prevent scaling.

The university's main objective is consolidating academic operations into a unified platform that eliminates fragmentation, enables student self-service to reduce registrar workload, and provides administrative visibility for planning and compliance. Success from the client's perspective means registrar staff time spent on manual tasks is reduced by 50%, professors adopt the grade recording system for 90%+ of their grading, students achieve 80%+ adoption within three months, and the system demonstrates 99.5% uptime reliability.

The client values modern technology that will remain relevant for 5+ years, emphasizing the quality and completeness of the MVP over ambitious feature lists that would compromise delivery. They prefer managed cloud hosting (Supabase) over on-premise infrastructure, eliminating the need to maintain servers and backups internally. Understanding client context ensures the solution aligns with institutional priorities and success metrics are relevant to actual business value.

2. Problem Statement

There is a lot of administrative clutter and inefficiency at the Metropolitan University of Tirana due to the academic administration's use of multiple overlapping systems, manual processes, and paper-based workflows. Institutional scalability, data quality, student experience, and efficiency are all negatively affected by this fragmentation.

2.1 Current Situation and Pain Points

2.1.1 Student Experience

Students at Metropolitan University face significant obstacles in accessing and managing their academic information. Without self-service access to academic progress, a student seeking to know their current GPA or understand their graduation timeline must submit a request to the registrar office and wait 24–48 hours for a response. Students submit assignments across multiple platforms and rely heavily on email, which makes it easy to lose or overlook submissions. There is no unified academic dashboard, forcing students to piece together information about their courses, grades, attendance records, and materials from multiple disconnected sources. More fundamentally, students lack real-time guidance on academic planning. Booking an appointment with an academic adviser currently requires waiting 5–7 days, and if the student has questions during that waiting period or needs quick clarification on course requirements, there is no way to get assistance. The financial aspect of student experience is equally opaque: students have no visibility into their tuition balance, payment deadlines, or payment history.

2.1.2 Professor Workflow

Professors must juggle multiple disconnected systems to manage their courses. Grades might be tracked in spreadsheets, attendance in paper roll calls or separate files, course materials scattered across Moodle, email, and personal drives, and assignments managed through various channels. Recording grades for a single class can take 30+ minutes due to manual data entry without validation or consistency checking.

Course roster management requires coordination with the registrar office for any enrollment adjustments. There is limited visibility into student progress until final exams, making it difficult to identify and support at-risk students early. The lack of integration between these systems means professors must manually synchronize information across platforms, introducing opportunities for errors and inconsistencies.

2.1.3 Administrative Operations

The registrar office and administrative staff operate without a unified system for user management, forcing them to manage student and professor accounts separately. Report generation is primarily manual, pulling data from disconnected databases and spreadsheets. Financial operations, including tuition, payments, and scholarships, manage themselves separately from the academic system, creating data boundaries that prevent comprehensive institutional analysis.

Audit trails are weak, making it difficult to track who changed what data and when, creating compliance risk for accreditation and institutional reporting. Enrollment management is manual and error-prone, with no automated workflows for common operations. As enrollment grows, these manual processes require proportional increases in administrative staff, making the system unable to scale efficiently.

2.2 Quantified Impact

The present situation has a measurable negative impact. The registrar’s office spends most of its time doing manual data entry and report generation. Differences in data between systems cause errors in student records that we need to fix by hand. Students wait for days for information they should receive instantly, which causes frustration and prevents prompt academic decisions. The requirement for corresponding increases in administrative staffing constrains scalability and leads to higher operational costs.

3. Solution Overview

M.I.A addresses these systemic challenges by consolidating all academic operations into a unified, modern, web-based platform. Rather than requiring users to navigate multiple disconnected systems, all stakeholders access a single source of truth tailored to their role and responsibilities.

3.1 Core Value Proposition

3.1.1 For Students

The M.I.A student portal transforms the academic experience from fragmented and reactive to unified and proactive. Students can view their complete academic profile in real time: current grades and GPA, attendance records by course, transcript for download, and clear progress toward graduation. Rather than waiting days for adviser appointments, students can open M.I.A and ask academic questions in natural language, receiving immediate personalized recommendations based on their major, completed credits, GPA, and graduation requirements.

The system maintains memory of student preferences across conversations. If a student mentions they prefer morning classes and are interested in artificial intelligence, M.I.A remembers this context and incorporates it into future recommendations. Financial information is transparent, showing tuition balance, payment schedule, and transaction history. All of this information is available 24/7, unlike human advisers.

3.1.2 For Professors

Professors gain a unified workspace replacing the current collection of disconnected tools. A single Grades interface handles both individual grade entry and bulk uploads for an entire class. Attendance is tracked and organized by course session. Assignments are created once and viewed with all student submissions in one place. Course materials are organized and easily shared with students.

Beyond consolidation, the system provides visibility. Grade distribution histograms show how the class performed on assignments. Attendance patterns highlight students missing multiple sessions. A course roster shows all enrolled students with their current grades and attendance record, enabling early identification of at-risk students. What currently takes 30+ minutes to record grades for a class can be done in 5 minutes through streamlined individual entry with validation.

3.1.3 For Administrators

Administrators transition from manual, disconnected operations to a unified institutional view. User management is centralized: create, edit, or delete accounts and assign roles from one interface. Financial operations are integrated with academic data, enabling comprehensive reporting on fee collection, payment status, and installment compliance. Administrators can configure tuition fees per major and have those fees automatically assigned to new students. Institutional reporting becomes automated rather than manual, with reports on enrollment, grade distribution, payment status, and course utilization generated on demand.

3.2 Success Criteria and Metrics

Success for M.I.A is measured across functional, performance, adoption, and business dimensions. Functionally, all three user portals must operate reliably with zero data integrity issues. Students must be able to view grades, submit assignments, and chat with M.I.A without errors. Professors must be able to record grades and manage attendance. The ability to manage users and generate reports is essential for administrators.

Performance targets are specific and measurable. Page load time must be under 3 seconds. API response time (95th percentile) must be under 500 milliseconds. LLM inference time must be under 2 seconds. System uptime must reach 99.5%, meaning less than 3.5 hours of downtime per year. These targets ensure the system feels responsive and reliable.

Adoption metrics measure whether users actually embrace the system. Within 3 months of launch, target 80% of students using the student portal at least once per week. Target 90% of professors using the grade recording interface instead of alternative methods. Expect 100% administrative engagement immediately. These adoption rates indicate users find the system valuable and trustworthy.

Business impact is measured through efficiency gains and user satisfaction. Target 50% reduction in registrar office time spent on manual data entry through automation. Target 80% of professor grade recordings accomplished in 5 minutes or less through bulk upload capability. Target M.I.A satisfaction rating of 4.0 out of 5.0 from student users. Measure fee collection improvement as students gain visibility into their payment obligations and can pay online.

These success criteria are specific, measurable, and achievable. They reflect the tangible improvements M.I.A brings to institutional operations and user experience.

4. Requirements Analysis

4.1 Functional Requirements

Functional requirements describe the services that the M.I.A university management system must offer. They are subdivided by access level: Admin, Professor, Student, and Public.

Req#	Description	Access Level	Rate
FR_01	User authentication is done through login and password, with JWT token issuance.	All Roles	High
FR_02	Role-based redirect after login: Admin, Professor, or Student dashboard.	All Roles	High
FR_03	Protected routes that block access to dashboards without a valid session token.	All Roles	High
FR_04	Create, read, update, and delete user accounts for students and professors.	Admin	High
FR_05	Assign and modify user roles across the system.	Admin	High
FR_06	View all system records: courses, grades, attendance, and submissions.	Admin	High
FR_07	Configure tuition fee amounts per academic major.	Admin	High
FR_08	Auto-assign fee records and installment plans to students upon enrollment.	Admin	High
FR_09	Record manual payments against student fee records and update settlement status.	Admin	Medium
FR_10	Create, update, and delete courses and upload associated course materials.	Professor	High
FR_11	Enroll students into courses and manage class rosters.	Professor	High
FR_12	Record and update grades for individual students per course or assignment.	Professor	High
FR_13	Record, update, and view student attendance per course session.	Professor	High
FR_14	Create, update, and delete assignments (homework, projects, exams) with due dates.	Professor	High
FR_15	View enrolled courses for the current semester.	Student	High
FR_16	View grades per course and live GPA.	Student	High
FR_17	View the attendance record broken down per course.	Student	High
FR_18	Access and download academic transcript.	Student	High
FR_19	View and submit assignments and projects with file upload tracking.	Student	High
FR_20	View tuition fee balance, installment schedule, payment status, and transaction history.	Student	Medium
FR_21	Access the M.I.A conversational AI adviser exclusively from the student dashboard.	Student	High

FR_22	Ask natural language academic questions and receive AI-generated answers.	Student	High
FR_23	Receive personalised course recommendations based on major and academic progress.	Student	High
FR_24	Track graduation timeline and credit hour progress through the AI adviser.	Student	Medium
FR_25	Student preferences and goals persist across M.I.A chat sessions.	Student	Medium
FR_26	Full multi-turn conversation history is maintained and used for context-aware replies.	Student	High
FR_27	Public landing page introducing M.I.A and the university portal.	Public	Low
FR_28	Public login page as the sole entry point for all user roles.	Public	High

4.2 Non-Functional Requirements

Non-functional requirements specify criteria that can be used to judge the operation of the system as a whole rather than specific behaviours. They describe emergent properties such as security, performance, usability, and reliability. Unlike functional requirements that teams can sometimes work around, non-functional requirements are essential for delivering a trustworthy and usable system.

For M.I.A, the most important non-functional requirements include security, performance, usability, reliability, and maintainability.

Security

M.I.A handles sensitive academic and financial data, making security a primary concern. Unauthorised access to student records, grades, or financial information is not permissible. All user passwords must be stored as hashed values using a modern algorithm such as bcrypt - plaintext storage is forbidden.

All API routes must be protected by JWT-based authentication. Requests without a valid token must receive an HTTP 401 response. Role-based access control must be enforced at the API layer independently of any frontend restrictions, ensuring a student cannot access admin or professor endpoints even if they manipulate the client.

All external credentials such as database connection strings, Supabase keys, Groq/Gemini API keys, and JWT secrets must be stored in environment variables and must never be committed to source control.

Performance

M.I.A is a multi-user system that must serve concurrent sessions for administrators, professors, and students simultaneously. Dashboard pages must load within an acceptable time under normal network conditions using the asynchronous capabilities of FastAPI.

The M.I.A AI adviser must return the first response token promptly, using streaming where supported, to minimise perceived latency and preserve a conversational feel. Database queries must leverage indexed foreign keys, particularly student_id and course_id, to avoid full-table scans as the volume of records grows.

Usability

The system must provide distinct graphical interfaces for each role: Admin, Professor, and Student. Each portal must have a visually clear layout so users can immediately identify their context and locate their features without confusion.

All interfaces must be user-friendly and simple to learn, with intuitive workflows. The student portal in particular, including the M.I.A chat interface, must be accessible to users with no prior knowledge of the system. The M.I.A chat must display conversation history as a standard threaded chat with visually separated user and assistant messages.

All interfaces must be responsive and adapt to different screen sizes across desktop and mobile devices, implemented via Tailwind CSS utility classes.

Reliability

The system must use Supabase-managed PostgreSQL, which provides automatic database backups and platform-level uptime guarantees. Student academic and financial records must be protected against data loss.

If the external LLM API (Groq or Gemini) becomes unavailable, the system must display a clear and graceful error message within the student portal without crashing or affecting other portal functionality. The AI adviser's unavailability must not degrade the core academic management features.

Maintainability

The codebase must follow a feature-branch Git workflow. Direct commits to the main branch are not permitted; all changes must be submitted via pull request and reviewed by at least one other team member before merging.

All commit messages must conform to the Conventional Commits format using the prefixes feat., fix., docs., or chore. to ensure a clear and traceable project history.

The backend must separate logic by domain into dedicated routers, auth, admin, professor, and student, so that each area can be developed, tested, and modified independently without risk of unintended side effects on other modules.

4.3 System Requirements

System requirements define the technical and architectural constraints within which M.I.A must operate. They specify the infrastructure, technology stack, and operational environment that underpin all functional and non-functional capabilities.

Req#	Description	Rate
SR_01	The system shall be built as a full-stack web application accessible via a standard web browser with no client-side installation required.	High
SR_02	The backend shall be implemented in Python using FastAPI, exposing a RESTful API consumed by the React frontend.	High
SR_03	The frontend shall be implemented in React with Tailwind CSS and communicate with the backend via a configurable base URL environment variable.	High

SR_04	The database shall be PostgreSQL hosted on Supabase, using a relational schema with normalised tables for users, courses, grades, attendance, assignments, and chat history.	High
SR_05	The AI adviser shall connect to an external LLM API (Groq or Gemini) configurable by environment variable, with no code changes required to switch providers.	High
SR_06	All external credentials (database URLs, API keys, JWT secrets) shall be stored in environment variables and excluded from version control via .gitignore.	High
SR_07	The system shall support concurrent sessions for multiple users across all three roles simultaneously.	Medium
SR_08	The application shall be runnable locally via uvicorn (backend) and npm run dev (frontend) with setup documented in the README.	Medium

4.4 User Stories

the system requirements from the perspective of end users. They capture the goals and expectations of each stakeholder group and provide a user-centered view of the system's functionality.

4.4.1 Student User Stories

The student portal is designed to provide students with easy access to their academic information and learning activities.

Progress Monitoring:
As a Student, I want to view my current grades and GPA in real-time so that I can stay informed about my academic standing.

Attendance Transparency:
As a Student, I want to check my attendance records for each course so that I can ensure my participation is being tracked correctly.

Credential Access:
As a Student, I want to download my academic transcript so that I can provide proof of my studies for external opportunities.

Task Submission:
As a Student, I want to view and submit my assignments through the portal so that I can keep track of deadlines in one place.

4.4.2 Professor User Stories

The professor portal supports teaching activities, student assessment, and course management.

Course Management:
As a Professor, I want to upload and manage my course materials so that students have digital access to the syllabus and resources.

Academic Assessment:
As a Professor, I want to record and update student grades so that students can see their performance in real-time.

Attendance Tracking:
As a Professor, I want to manage student attendance records so that I can monitor participation and meet institutional requirements.

Assignment Distribution:
As a Professor, I want to create projects and assignments with specific due dates so that I can organize the curriculum effectively.

4.4.3 Administrator User Stories

The administrator portal enables centralized management of users, roles, and institutional information.

Account Creation:
As an Administrator, I want to create new student and professor accounts so that new members of the university can access the system.

Role Management:
As an Administrator, I want to assign and update user roles so that I can control access permissions as staff responsibilities change.

System Oversight:
As an Administrator, I want full access to all institutional records and courses so that I can maintain a single source of truth for the entire university.

4.4.4 M.I.A. Academic Adviser User Stories

The M.I.A conversational adviser provides personalized academic assistance and guidance to students through natural language interactions.

Instant Guidance:
As a Student, I want to ask M.I.A academic questions so that I do not have to wait for an appointment with a human adviser.

Curriculum Planning:
As a Student, I want M.I.A to recommend courses based on my major and academic progress so that I can make informed choices about my next semester.

Graduation Tracking:
As a Student, I want M.I.A to track my credit hours and graduation timeline so that I know exactly when I am eligible to graduate.

Contextual Memory:
As a Student, I want the AI adviser to remember my goals and preferences across different chat sessions so that I do not have to repeat my background information every time.

5. System Design

5.1 BPMN Process Diagrams

This chapter presents the three BPMN (Business Process Model and Notation) process flow diagrams that document the key workflows in the M.I.A. system. These diagrams use swimlane notation to show which system component or actor is responsible for each step in the process.

5.1.1 Authentication and Login Flow

The Authentication and Login BPMN diagram models the user login process across two swimlanes: the Browser (frontend) and the FastAPI backend. The process begins at the Login Page where the user enters email and password. The frontend sends a POST request to the backend. The backend swimlane shows a gateway that validates credentials: if invalid, an error message is returned; if valid, the flow continues through three sequential tasks: Validate Credentials, Verify bcrypt Hash, and Generate JWT Token (8-hour expiry). The token is returned to the browser, which stores it in localStorage. A second gateway on the frontend branches by role, redirecting to the Admin Dashboard, Professor Dashboard, or Student Dashboard respectively.

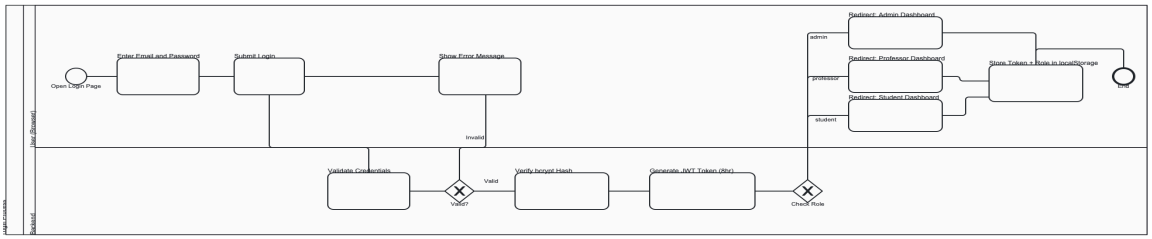

Figure 1: Authentication and Login Flow - credential validation, JWT issuance, and role-based redirection

5.1.2 Assignment Submission and Grading

The Assignment Submission and Grading BPMN diagram spans two swimlanes: Professor and Student. The Professor lane shows the creation flow: the professor creates an assignment (title, course, due date, type) followed by a gateway checking whether a file attachment is needed. If yes, the file is uploaded to Supabase Storage and the URL is saved with the assignment record. If no, the assignment is saved directly to the database. An intermediate event marks the assignment as Published. The Student lane activates upon the Published event and shows the student viewing the assignment, selecting a submission type (file or text), and submitting via POST /assignments/{id}/submit. An intermediate event marks the submission as received. Control returns to the Professor lane where the professor opens the submission, adds a grade and feedback, and saves the grade to the database. The process ends with a final end event in the Professor lane.

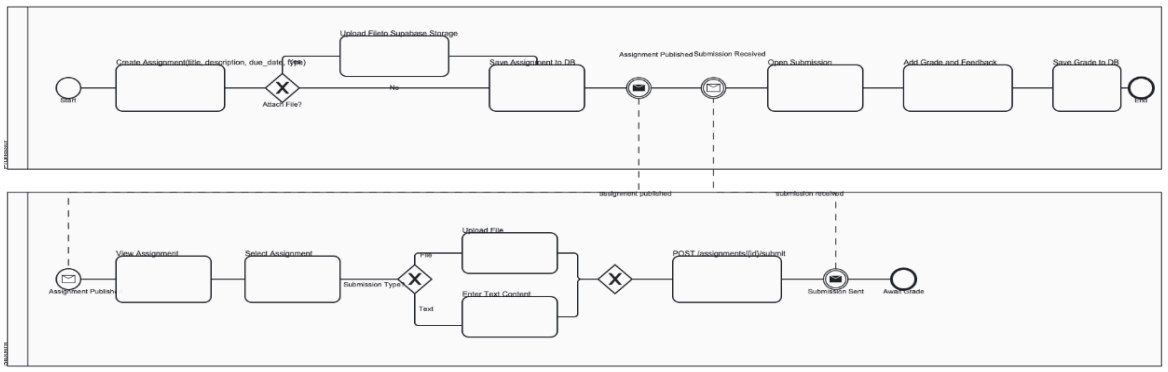

Figure 2: Assignment Submission and Grading - creation, publication, student submission, and professor grading

5.1.3 M.I.A Conversational Flow

The M.I.A. Conversational Flow BPMN diagram covers the full AI adviser interaction across two swimlanes: the Student and the FastAPI MIA Agent. The process begins with the student opening the M.I.A. chat interface. A gateway checks whether an existing session exists: if yes, the existing session is loaded; if no, a new chat session is created. The student types and submits a question. The MIA Agent lane shows a sequence of context-building tasks: Fetch Student Profile (courses, enrollment), Fetch Grades and GPA, Fetch Attendance Statistics, Fetch Preferences, and Prepare Message to LLM (system prompt + context + history + current message). The message is sent to the Groq API. A gateway checks whether the AI response is satisfactory: if not, the response is discarded and the context rebuilt; if yes, the response is stored as a message record and returned to the student. The student can continue the conversation, which loops back to the message submission step, or end the session.

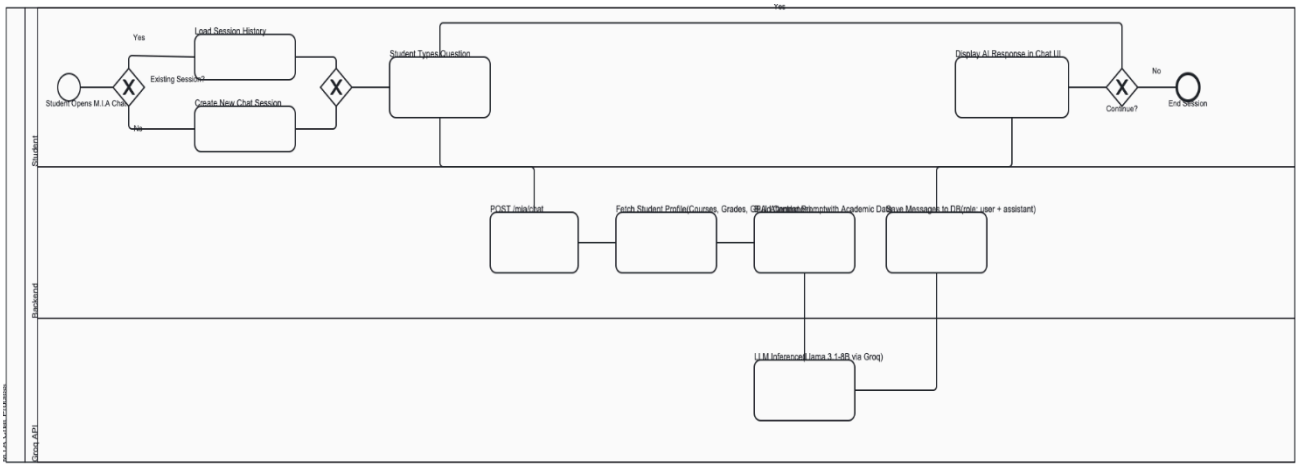

Figure 3: M.I.A. Conversational Flow - session management, context assembly, LLM invocation, and response persistence

5.2 Data Flow Diagrams (DFD)

5.2.1 DFD Level 0 – System Context

The Level 0 Data Flow Diagram presents the M.I.A. system as a single process interacting with its external environment. Four external entities communicate with the system: three human actors - Student, Professor, and Admin - and one external system, the Groq API, which powers the AI adviser functionality. Students send credentials, chat messages, and assignment submissions, and receive grades, GPA data, and AI-generated responses in return. Professors submit grade values, attendance records, and course materials, receiving back enrollment lists and course reports. Administrators manage users, course configurations, and fee structures, and receive payment reports and system confirmations. The Groq API operates in a request-response pattern exclusively initiated by the system, receiving a compiled student academic profile alongside conversation history and returning a natural language completion. Notably, no external payment gateway appears at this level, as the finance module is fully admin-driven and internally reconciled within the system boundary.

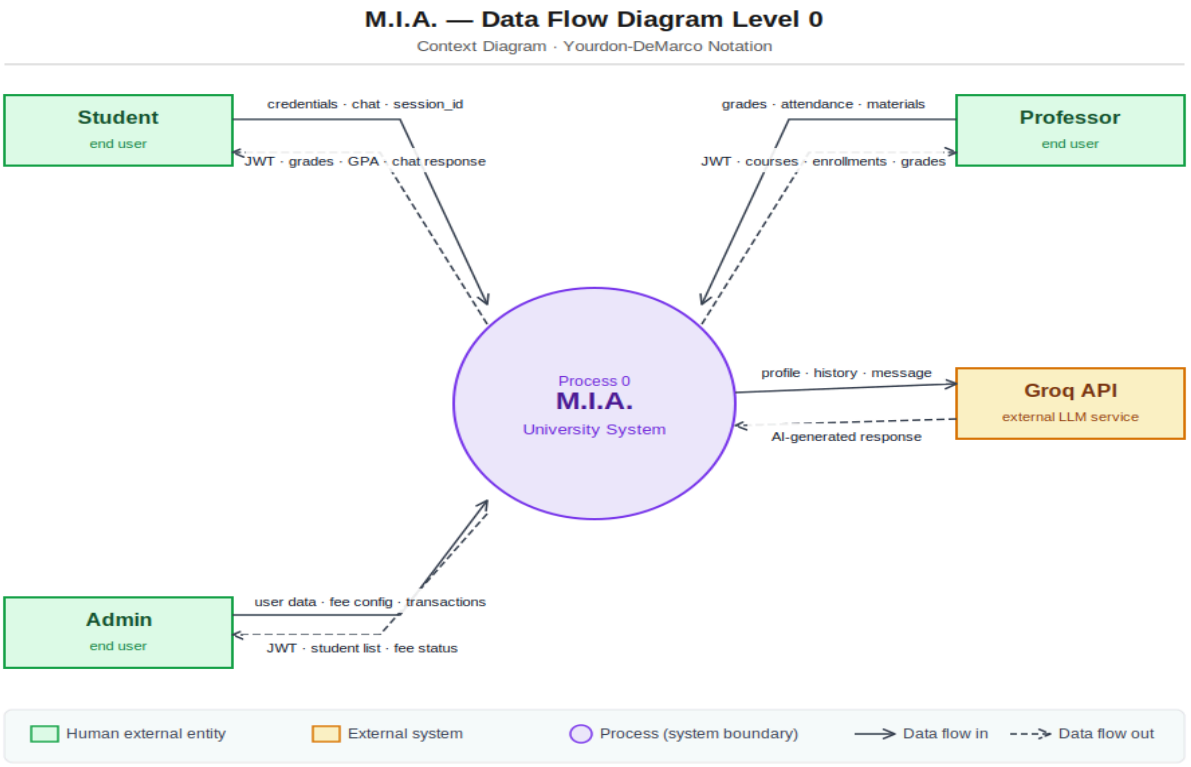

Figure 4: DFD Level 0 - M.I.A. system context showing all external entities and top-level data flows

5.2.2 DFD Level 1 – Internal Processes

The Level 1 Data Flow Diagram decomposes the central M.I.A. process into eight distinct sub-processes, each corresponding to a functional module of the system. Process 1.0 (Authentication) handles login, registration, and JWT issuance for all three human roles. Process 2.0 (User and Course Administration) manages the full lifecycle of users and courses under admin control. Process 3.0 (Enrollment Management) handles the assignment of students to courses, connecting exclusively to the Admin entity. Processes 4.0 (Grade Management) and 5.0 (Attendance Tracking) serve all three human roles with read access and accept write input from Professors. Process 6.0 (Assignments and Materials) manages file-based interactions, interfacing with Supabase Storage as an external file store for upload and URL retrieval. Process 7.0 (Payment Processing) handles the full fee lifecycle, from major fee configuration to installment auto-generation and transaction reconciliation, without any external payment gateway. Process 8.0 (MIA Chat) is the only process that communicates with the Groq API, compiling a rich student academic context from six data stores before dispatching each request. The diagram also highlights D14 (student_preferences) with a dashed border, indicating a data store that is read by the system but has no write endpoint in the current implementation.

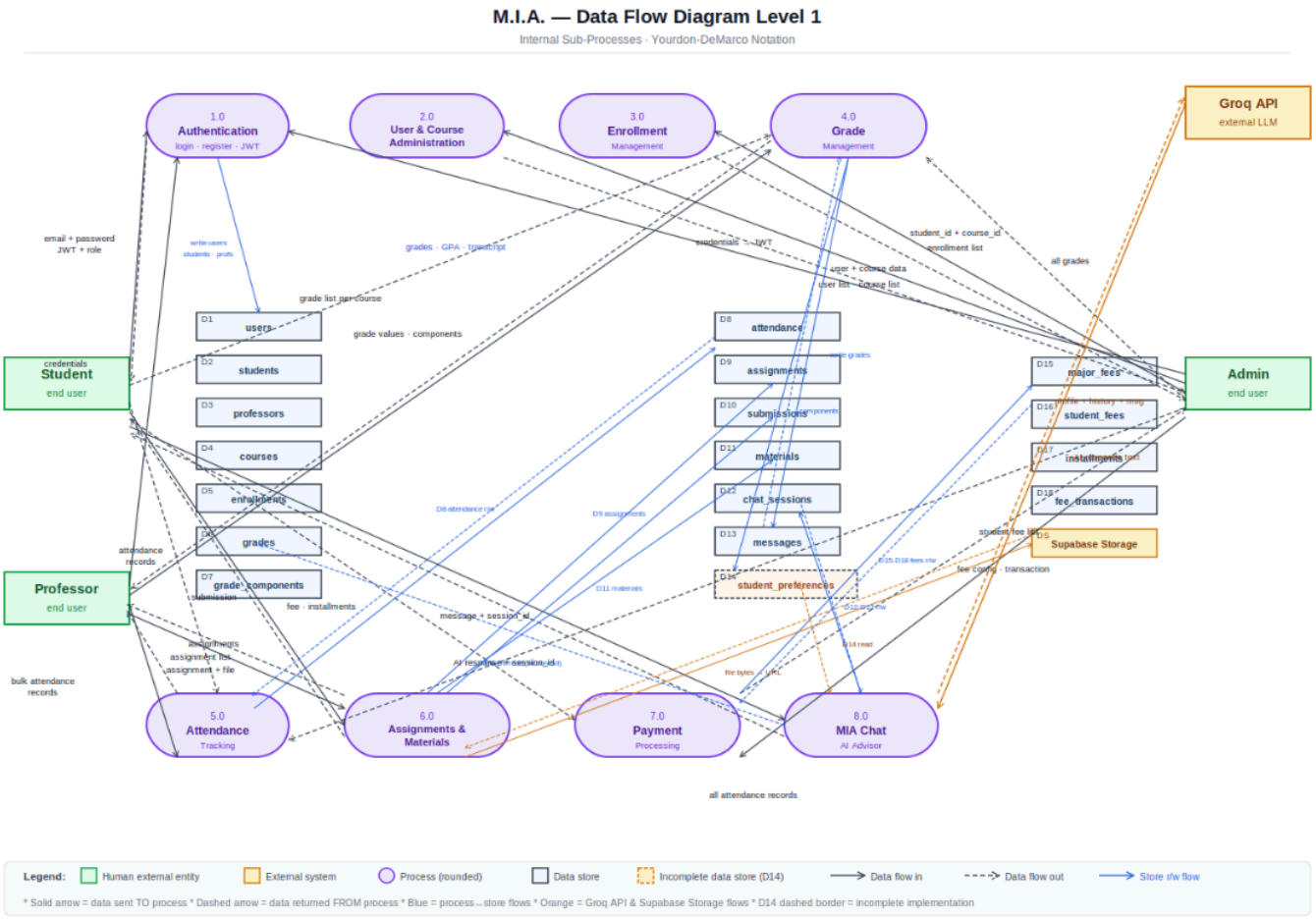

Figure 5: DFD Level 1 - internal sub-processes, 18 data stores, and all inter-component data flows

5.3 Behavioral Diagrams

This section presents the behavioral models of the M.I.A. system through UML Activity Diagrams and State Diagrams. Activity Diagrams capture process flows with decision points and branching logic. State Diagrams model the lifecycle of key domain objects.

5.3.1 Use Case Diagrams

5.3.1.1 Student Portal Use Cases

The Student Portal Use Case Diagram captures the full scope of student interactions with the system. The student actor connects to eight primary use cases: Chat with M.I.A (which extends to the AI Agent sub-system for answering questions, remembering preferences, recommending courses, and tracking graduation), Track Payments, View Enrolled Courses, Check Grades and GPA, View Attendance, Submit Assignments, View Tuition Balance, and Access Student Profile. Several of these use cases include relationships to shared system data and extend to lower-level operations.

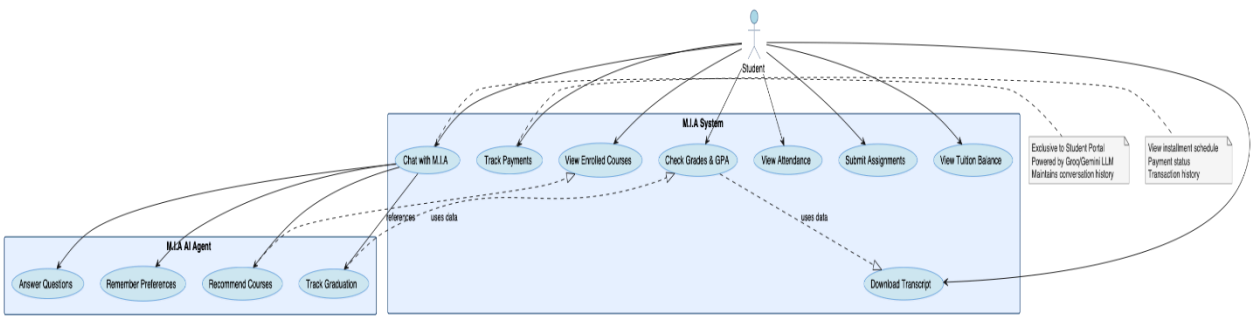

Figure 6: Student use cases - view grades, submit assignments, access M.I.A chat

5.3.1.2 Professor Dashboard Use Cases

The Professor Dashboard Use Case Diagram shows the professor actor interacting with five primary use cases: Upload Course Materials (which connects to Supabase Storage for file management and makes materials available to students), Create Course, Manage Assignments, Record Grades, and Track Attendance. Three secondary use cases are linked through UML relationships: Enroll Students (required by Create Course), View Student Submissions (related to Manage Assignments), and View Class Roster (which uses data from attendance and grades)

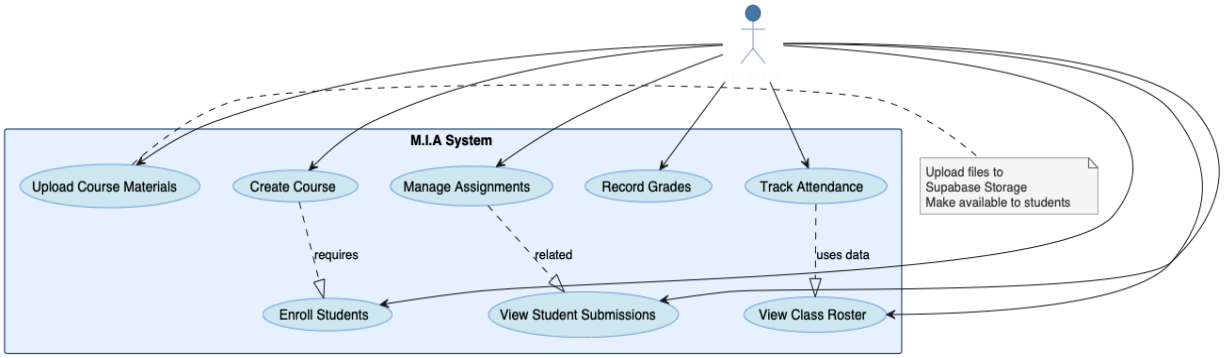

Figure 7: Professor use cases - manage courses, record grades, track attendance

5.3.1.3 Administrator Portal Use Cases

The Administrator Portal Use Case Diagram covers the administrative domain. The admin actor connects to six primary use cases: Manage Tuition Fees, Create Users, Update User Info, Delete Users, View System Reports, and Track Payments. Two secondary use cases extend the admin scope: Configure Payment Plans (included in Manage Tuition Fees) and Assign Roles (required by all user management operations). A note on the diagram indicates that fee management extends beyond the original project scope to include automatic student assignment by major.

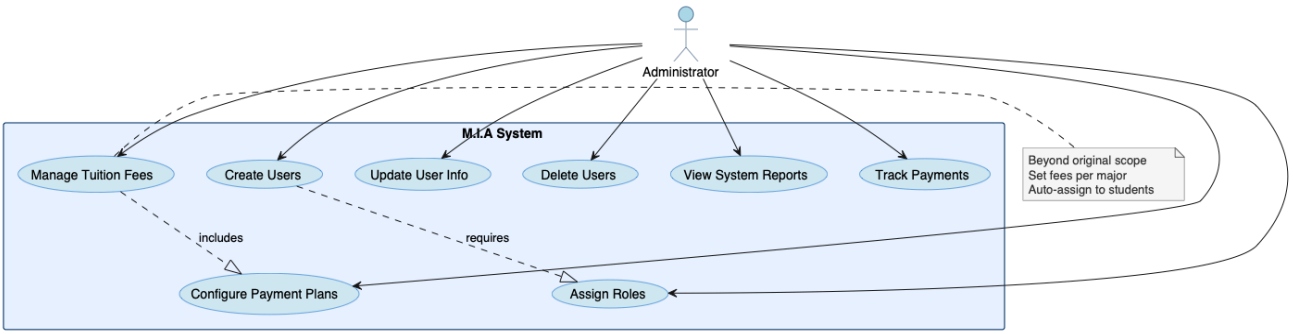

Figure 8: Administrator use cases - user management, system oversight, financial operations

5.3.2Activity Diagrams

5.3.2.1 Login Activity

The Login Activity Diagram models the authentication workflow from the moment a user opens the login page through JWT issuance and role-based redirection. The flow begins with the user entering email and password. The system submits credentials to POST /auth/login, where FastAPI validates the request format via Pydantic schema. A decision node checks whether the credentials are valid: if not, an error message is returned and the user is prompted to retry. If valid, the system looks up the user record, verifies

the bcrypt password hash, and generates a signed JWT with an 8-hour expiry. The JWT, role, and name are stored in localStorage. A second decision node branches on role: Admin users are redirected to the Admin Dashboard, Professors to the Professor Dashboard, and Students to the Student Dashboard. The flow terminates at the dashboard for the authenticated role.

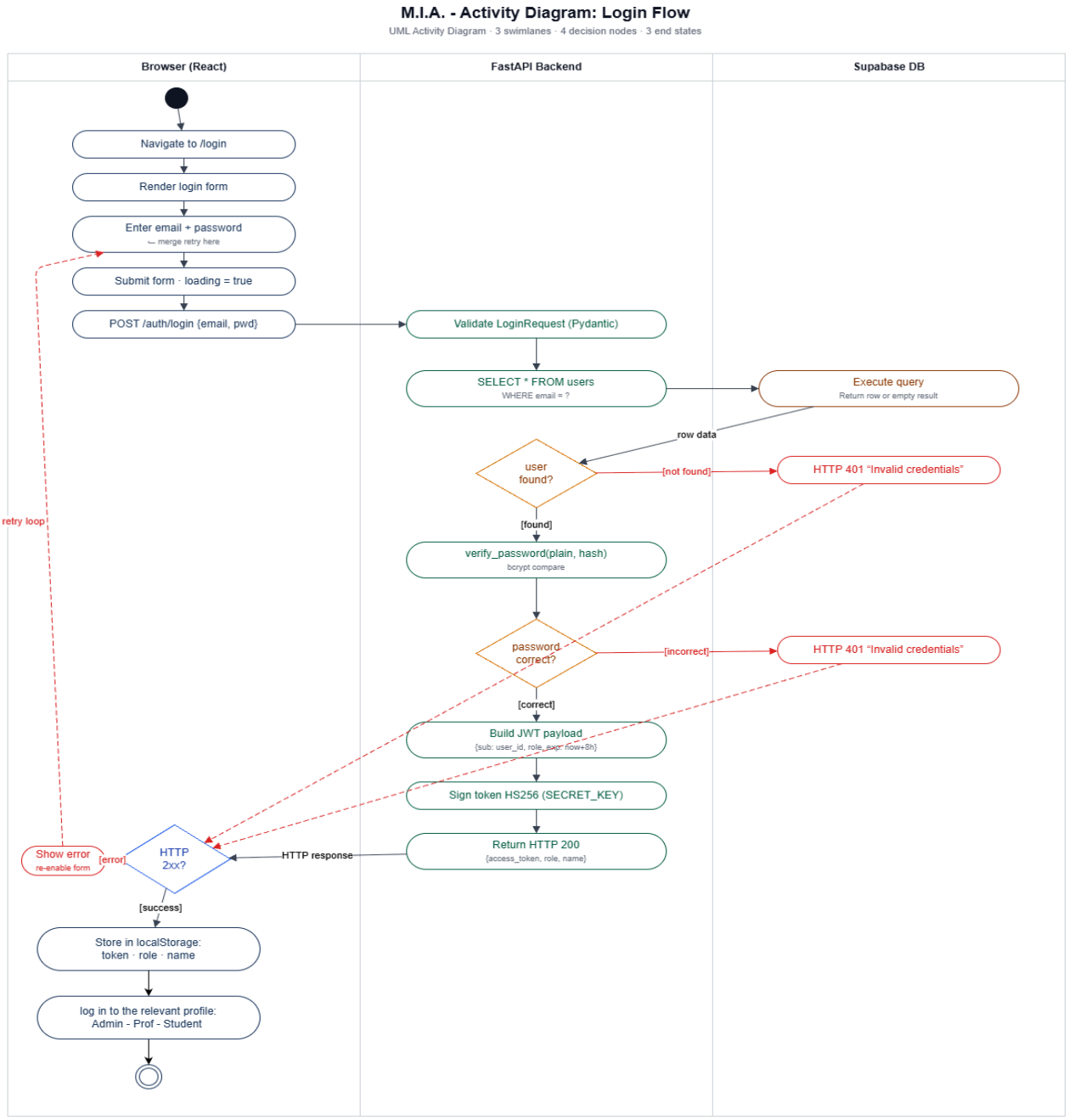

Figure 9: Login Activity Diagram - credential validation, JWT issuance, and role-based redirection

5.3.2.2 Grade Recording Activity

The Grade Recording Activity Diagram models the professor's workflow for entering grades, which is one of the most structurally complex flows in the system due to the validation logic for grade component weights. The flow begins with the professor selecting a course from their dashboard. A decision node checks whether grade components have been defined for the course: if not, the professor must first create components (name + weight), with the system validating that the total weight across all components does not exceed 100%. Once components exist, the professor selects a student from the enrolled roster and enters a grade value for a specific component. The system validates that the value is between 0 and 100 and that the semester matches the enrollment record. If validation fails, an error is returned, and the

professor is prompted to correct the entry. If valid, the grade record is written to the database with the component name and weight denormalized into the grade row for historical preservation. The flow allows the professor to continue entering grades for additional students before completing the session.

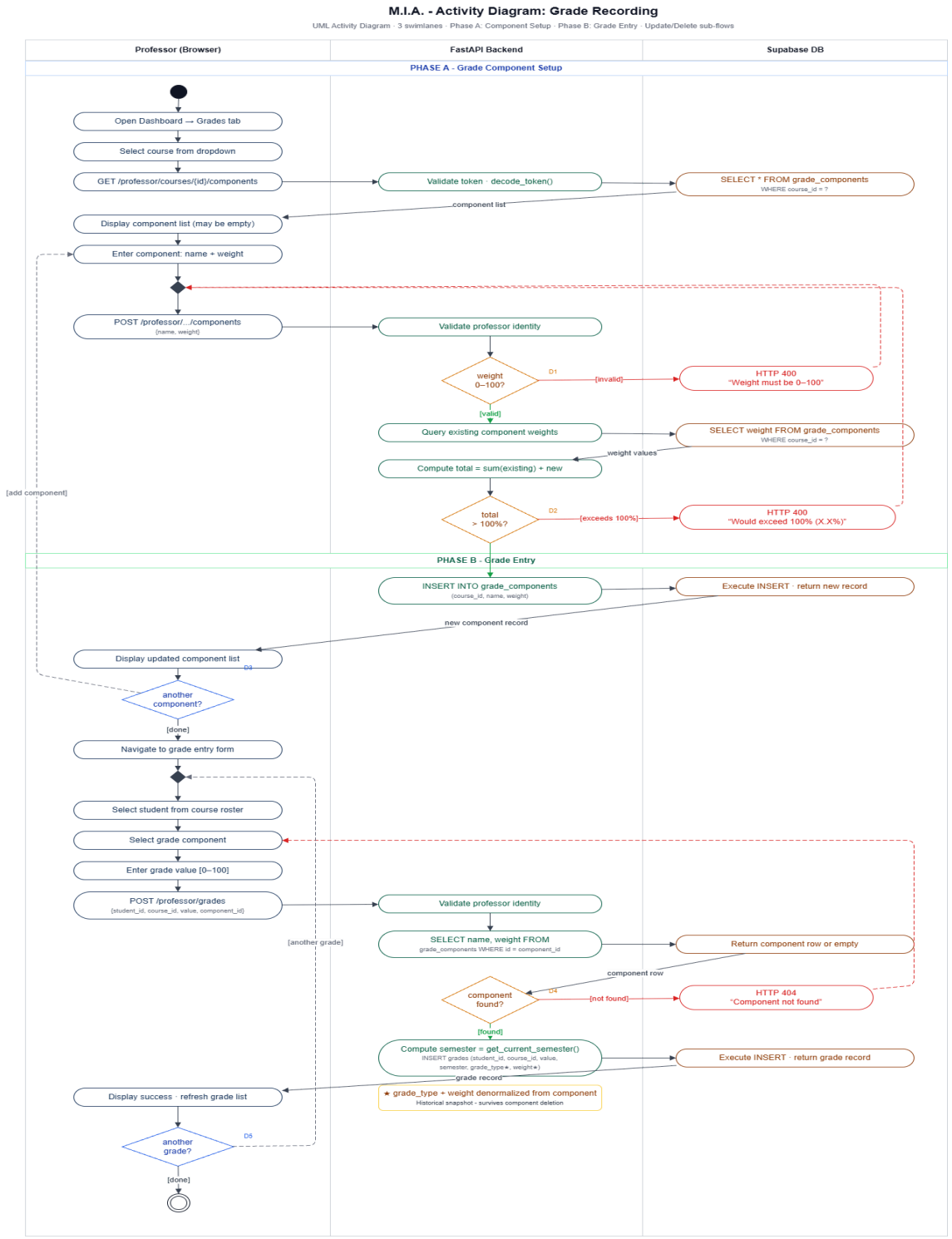

Figure 10: Grade Recording Activity Diagram - component weight validation and denormalized grade storage

5.3.2.3 M.I.A. Chat Activity

The MIA Chat Activity Diagram models the full request-response cycle of the AI adviser, from the student sending a message through Groq API invocation and response storage. The flow begins with the student opening the chat interface. A decision node checks whether an existing session_id is provided: if yes, the existing session is loaded and its message history retrieved from the messages table; if no, a new chat_session record is created. The student types and submits a message. The MIA Agent in FastAPI validates the JWT, then executes a multi-step context assembly: it fetches the student's enrolled courses, grades with GPA calculation, attendance statistics, and student preferences from five separate data store reads. The assembled context, system prompt, conversation history, and current message are packaged into a request to the Groq API. The Groq API returns an AI-generated response text. The user message and AI response are both persisted to the messages table. The response is returned to the frontend and displayed in the chat interface. The student can continue the conversation or end the session.

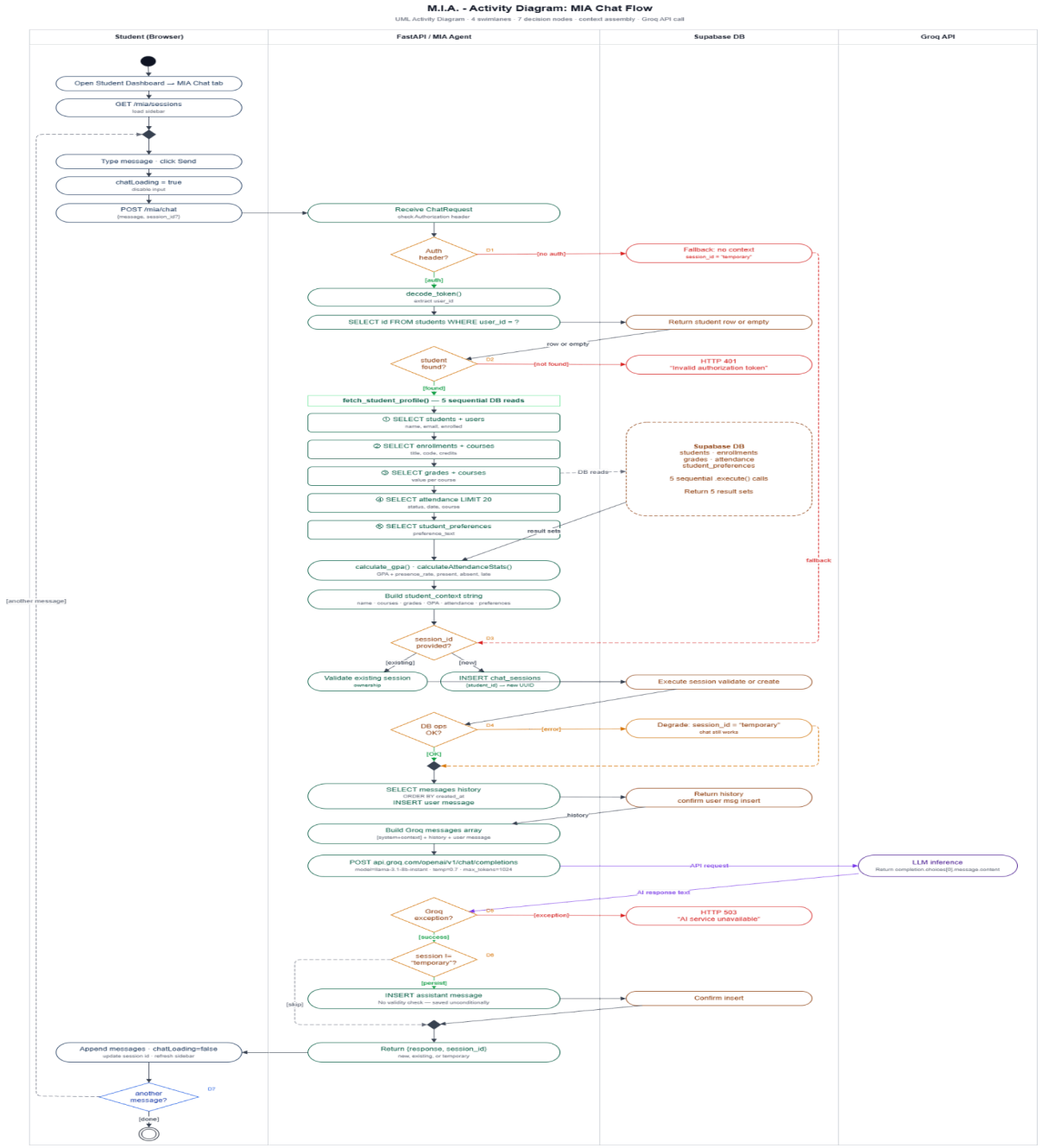

Figure 11: MIA Chat Activity Diagram - session management, context assembly, Groq invocation, and message persistence

5.3.3 State Diagrams

5.3.3.1 Assignment Lifecycle States

The Assignment Lifecycle State Diagram models the states an assignment record passes through from creation to final grading. An assignment begins in the Draft state when a professor creates it but before it is published. The transition to Published occurs when the professor confirms publication, making the assignment visible to enrolled students. From Published, the assignment moves to Submitted when a student submits a response via the submissions table. The final transition to Graded occurs when a professor adds a grade and feedback to the submission record. The diagram reflects a currently partial implementation: the Graded state exists in the schema (grade and feedback columns are present in the submissions table) but no write endpoint currently populates these fields, making the Submitted→Graded transition a planned but not yet implemented feature.

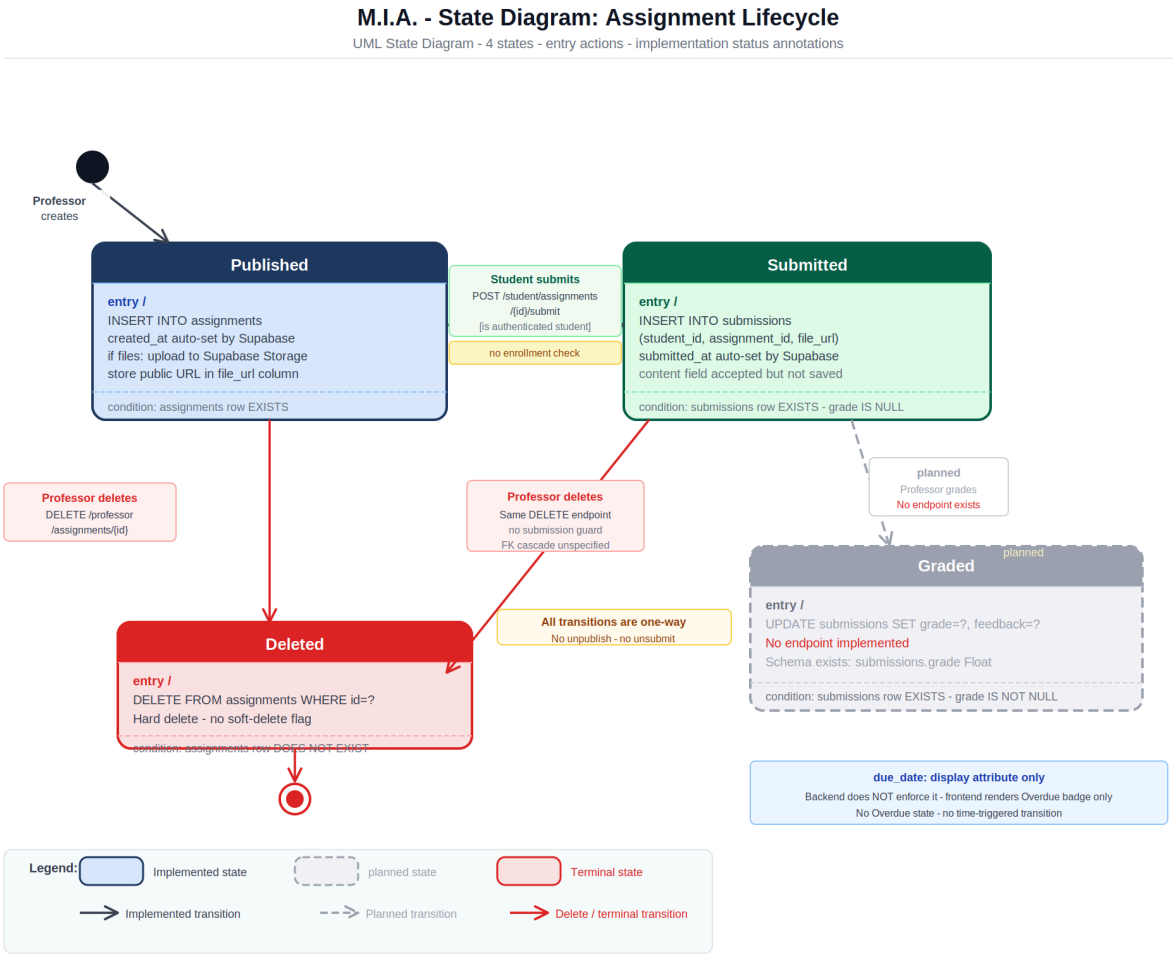

Figure 12: Assignment Lifecycle States - Draft → Published → Submitted → Graded

5.3.3.2 User Session Lifecycle

The User Session State Diagram models the lifecycle of authentication state in M.I.A. across three states, including one fewer than a naive reading of the architecture would suggest and one more than the codebase makes explicit. The **Unauthenticated** state is both the initial state and the state after logout. It is defined by the absence of a token key in localStorage. In this state the axios interceptor sends no Authorization header, and any protected backend endpoint returns HTTP 401. The **Authenticated** state is a single composite state with a role attribute, there are no separate sub-states per role. Upon successful login, three keys are written to localStorage in sequence: token (the HS256-signed JWT with an eight-hour absolute expiry), role (one of

"admin", "professor", or "student"), and name. The role attribute drives which dashboard component is rendered, which ProtectedRoute wrappers pass, and which backend endpoints are accessible. There is no role-switching mechanism within a session. Two transitions lead back to Unauthenticated: explicit logout, which calls localStorage.clear() and navigate('/login') with no backend HTTP request (the JWT remains cryptographically valid for the remainder of its window after logout), and ProtectedRoute guard failure, which fires a Navigate replace redirect when either the token is absent or the role string does not match the expected prop. Critically, ProtectedRoute performs only a truthy string check on the token as it does not validate the JWT signature or expiry.

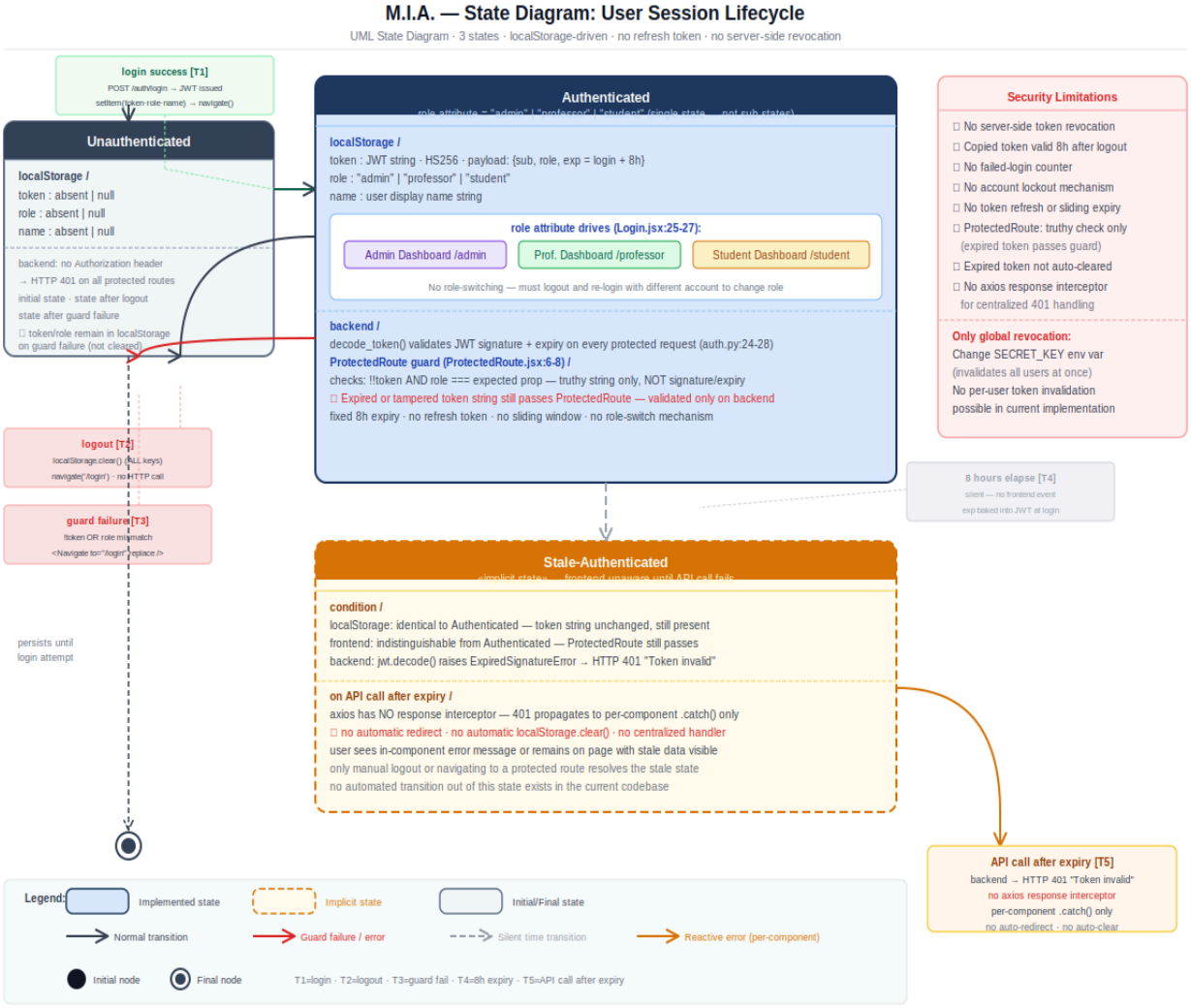

Figure 13: Payment Status States - Pending → Partial → Settled with scholarship modifier

5.4 Structural Diagrams

5.4.1 Class Diagram

The M.I.A. system's class structure is presented across two diagrams to preserve readability given the breadth of the schema.

Core Academic Diagram: The first diagram covers eleven classes organized around two central entities: Course and Student. User sits at the top of the hierarchy and holds a composition relationship with both Student and Professor, reflecting the Identity Map pattern used in the database, each role exists as a separate table linked to the shared users table via a foreign key, with no OOP inheritance involved. Course acts as the structural anchor for the academic domain, holding composition relationships with

GradeComponent, Assignment, and Material, all of which cannot exist independently of their parent course. Enrollment serves as an aggregation junction between Student and Course, where the student and course records persist independently and only the enrollment record is removed upon unenrollment. Grade and Attendance are modeled as plain associations from both Student and Course, reflecting their role as historical records that persist beyond the enrollment lifecycle. A dashed dependency arrow from GradeComponent to Grade documents the deliberate denormalization in the schema , grade type and weight are copied into each grade record at insert time to preserve historical accuracy. Submission is a composition child of Assignment, additionally associated with Student as the submitting party.

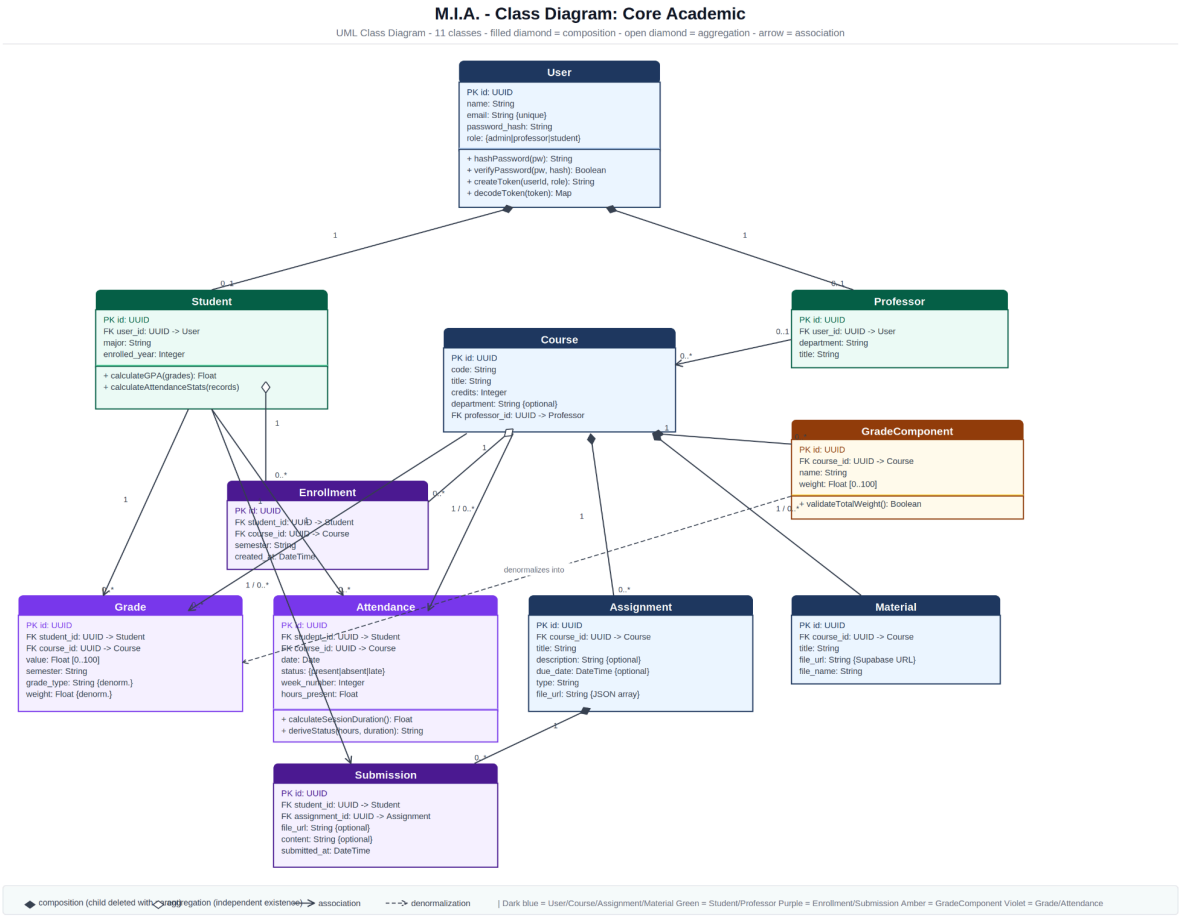

Figure 14: Core Academic Class Diagram - 11 classes with composition, aggregation, and association relationships

Finance Diagram: The second diagram covers the four finance classes centered on StudentFee, which acts as the root aggregate for a student's entire fee lifecycle. Installment and FeeTransaction are both composition children of StudentFee and are deleted when the parent record is removed. MajorFee is connected to StudentFee via a dashed dependency arrow labeled "derives major," explicitly indicating that no foreign key exists between them as the relationship is resolved at runtime through string equality on the major field. Student appears as an external boundary class with a dashed outline, referencing its definition in the Core Academic diagram without duplicating its attributes. The non-trivial methods on StudentFee such as computeStatus, applyScholarship, removeScholarship, and reconcileInstallments, are included because they encode business logic that is not derivable from the attributes alone and represents the most complex behavioral layer in the system.

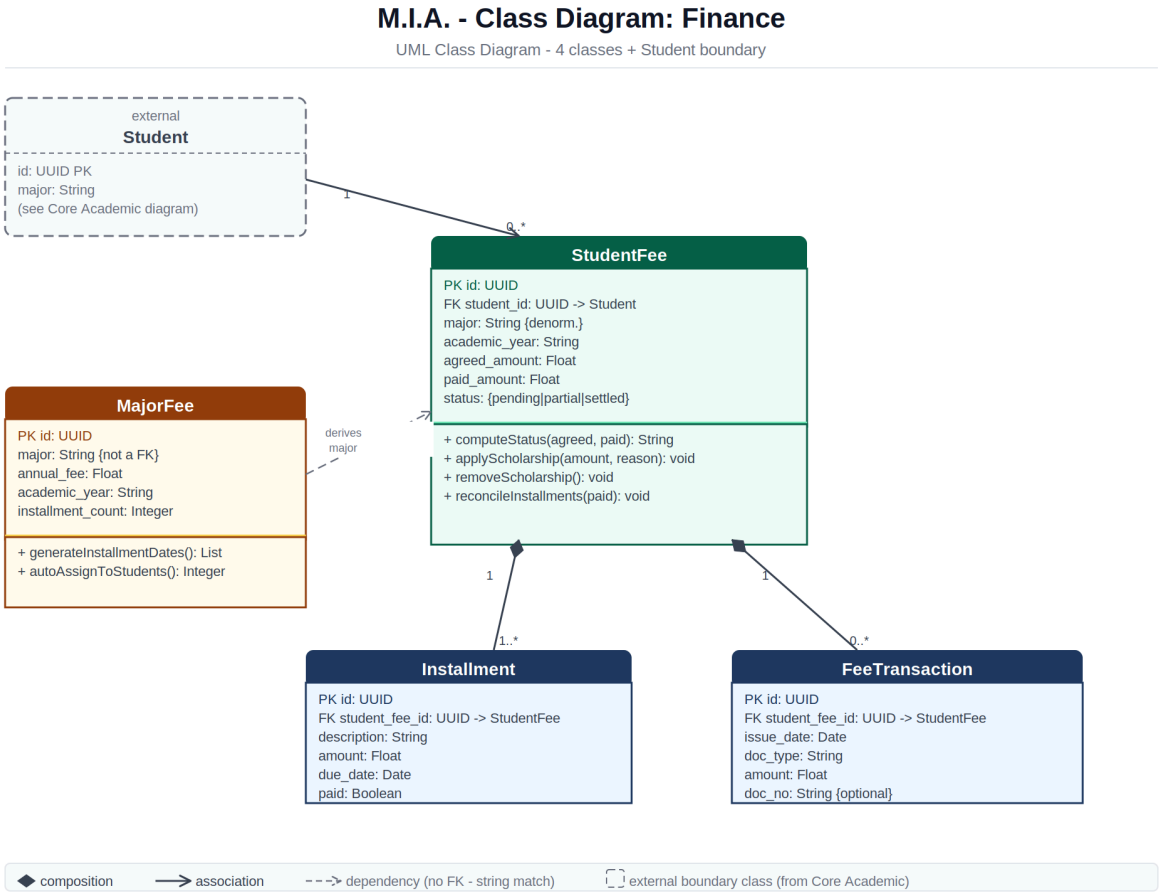

Figure 15: Finance Class Diagram - StudentFee as root aggregate with composition children and MajorFee dependency

5.4.2 Data Model (Database Schema)

The system's data model reflects the academic domain with 18 tables organized into four functional clusters. The identity cluster contains users, students, and professors, implementing an Identity Map pattern where each role-specific table holds a foreign key back to the shared users table. The academic cluster contains courses, enrollments, grades, grade_components, attendance, assignments, submissions, and materials. The finance cluster contains major_fees, student_fees, installments, and fee_transactions. The AI cluster contains chat_sessions, messages, and student_preferences.

Foreign key relationships enforce referential integrity throughout. Constraints ensure data validity: grades are validated between 0–100, grade component weights must sum to at most 100% per course, and payment status is computed automatically from agreed amount versus paid amount. Two structural anomalies are documented: the submissions table is written to by students but has no read endpoint in the current implementation, and the student_preferences table is read by the MIA agent but has no write endpoint.

5.4.3 Component Diagram

The Component Diagram presents the M.I.A. system across four deployment nodes, each drawn with a dashed boundary to reflect its runtime isolation.

The Browser node contains the React Frontend, a Vite-built single-page application served from Vercel's CDN or directly from FastAPI's static file handler. Internally it is organized into three layers: an API Client built on axios with a JWT interceptor that reads the token from localStorage and injects it as a Bearer header on every request; a Route Guard (ProtectedRoute.jsx) that enforces role-based access; and a

Pages layer comprising Login, Student Dashboard, Professor Dashboard, and Admin Dashboard. There is no React Context in the codebase - auth state is stored exclusively in localStorage. The frontend communicates with the backend exclusively through the API Client; it has no direct Supabase SDK dependency.

The Railway/Render node contains the FastAPI Backend as a single Python process. All six sub-components share the same app instance and Supabase client singleton defined in db.py. The Auth Router sits at the top of the internal hierarchy: every other router and the MIA Agent import decode_token() directly from auth.py. The MIA Agent is a distinct component because it has two responsibilities no other router shares: it assembles a rich student academic context from six data stores before each request, and it is the sole consumer of the Groq API. The Supabase Cloud node contains two separate components within the same managed project: the PostgreSQL database and the Supabase Storage bucket. The Groq Cloud node is marked as an external component, consumed only by the MIA Agent.

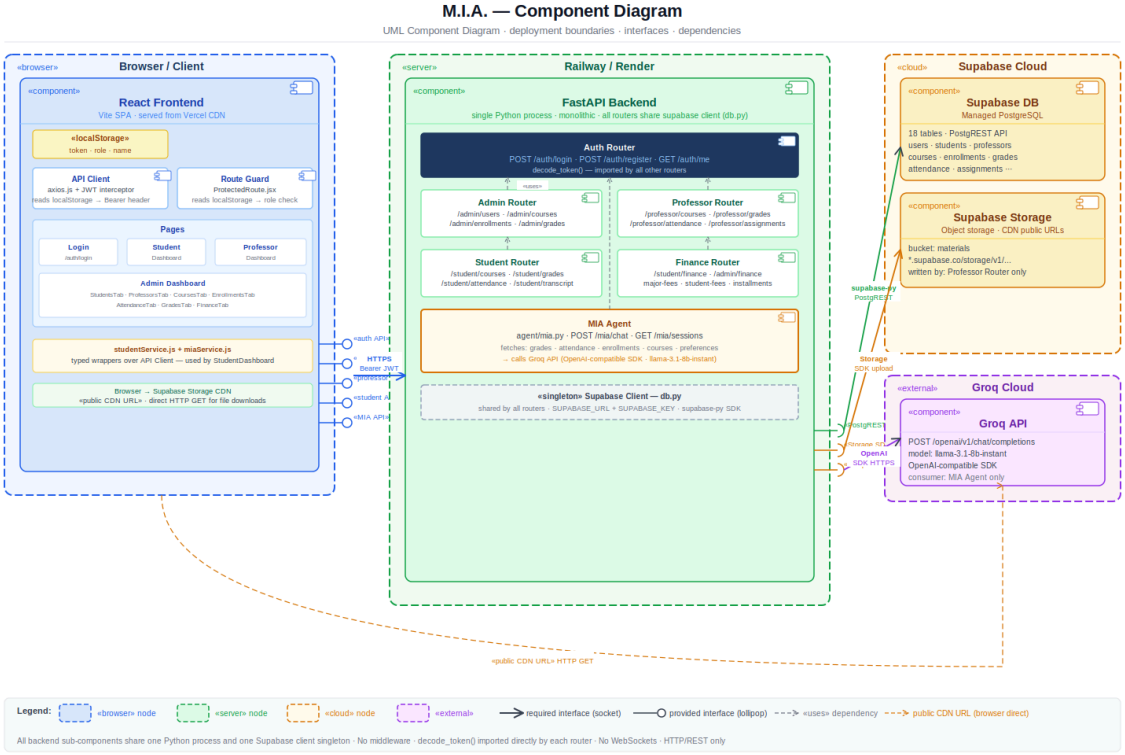

Figure 16: Component Diagram - four deployment nodes with interface connections and dependency relationships

6. System Architecture

6.1 Architecture Overview

M.I.A follows a modern three-tier architecture with an additional AI integration tier for the conversational adviser. This separation of concerns enables parallel development, independent scaling, and clear responsibility boundaries. The presentation tier consists of React applications compiled and deployed as static assets. The frontend communicates exclusively through REST APIs, making the system architecture clean and decoupled. The application tier is implemented in FastAPI and Python, handling request validation, business logic, authentication, and database operations. The data tier uses PostgreSQL as the relational database and Supabase Storage for file assets. The AI tier integrates with the Groq API for LLM inference powering the M.I.A. conversational adviser.

6.2 Layered Architecture Diagram

The M.I.A. system follows a five-layer architecture, though two of those layers are partially merged in practice due to the project's scale and design choices.

Layer 1 (Presentation) contains the entire React frontend running in the browser, encompassing all page components, the route guard, and the shared navigation bar. Layer 2 (HTTP Interface) is handled by FastAPI and consists of route declarations, Pydantic schema validation, CORS middleware configuration, and the SPA catch-all fallback route. This layer is responsible for rejecting malformed requests with HTTP 422 before any business logic is reached. Layer 3 (Application Logic) is where business rules and data queries coexist in the same router functions, following the fat router pattern common in small FastAPI projects. There is no service layer, no repository abstraction, and no command/query separation. The MIA Agent calls the Groq API directly from this layer, skipping Layer 4 entirely. Layer 4 (Data Client) is a single file: db.py, which instantiates the Supabase Python SDK client as a singleton. Layer 5 (External Services) contains Supabase PostgreSQL, Supabase Storage, and the Groq Cloud API.

Two deliberate layer violations are annotated in the diagram: the MIA Agent bypasses Layer 4 to reach Groq directly, and the browser bypasses Layers 2 through 4 entirely when downloading a file from a Supabase Storage public CDN URL.

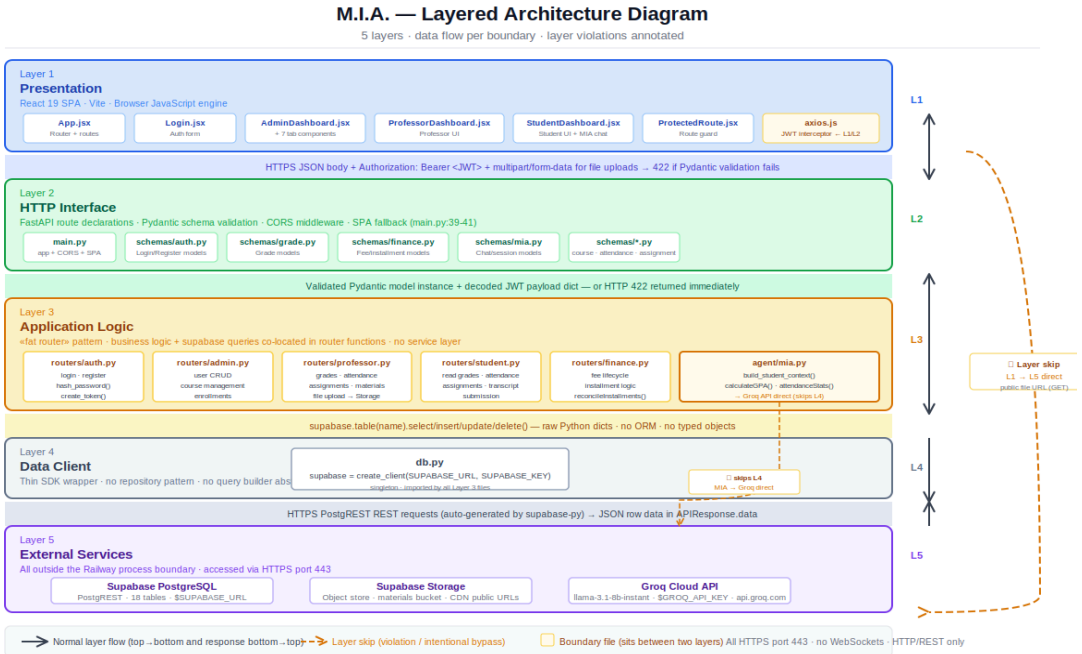

Figure 17: Layered Architecture - 5 layers with layer bypass annotations for MIA Agent and browser CDN access

6.3 Deployment Diagram

The Deployment Diagram shows four runtime nodes connected exclusively via HTTPS on port 443, with no WebSockets, no polling, and no webhooks anywhere in the system.

The Browser node runs the Vite-compiled React SPA, served as static assets by FastAPI from the backend/static/ directory. Authentication state is persisted in localStorage under three keys (token, role, name). The VITE_API_BASE_URL environment variable defaults to an empty string, making all API calls relative to the current origin and enabling the single-server deployment model. The Railway/Render node runs a single Python process: uvicorn main:app, launched from the backend/ directory. All six sub-components share this one process and one Supabase client singleton. The Supabase Cloud node contains two sub-components within the same project: the PostgreSQL database accessed via PostgREST, and Supabase Storage handling uploaded files in the materials bucket. The Groq Cloud node exposes an OpenAI-compatible endpoint serving the llama-3.1-8b-instant model, consumed solely by the MIA Agent. A dashed arrow annotated as a direct CDN bypass shows the one out-of-band connection: when a student opens a course material file, the browser issues a direct HTTPS GET to the Supabase Storage public URL without JWT and without FastAPI involvement.

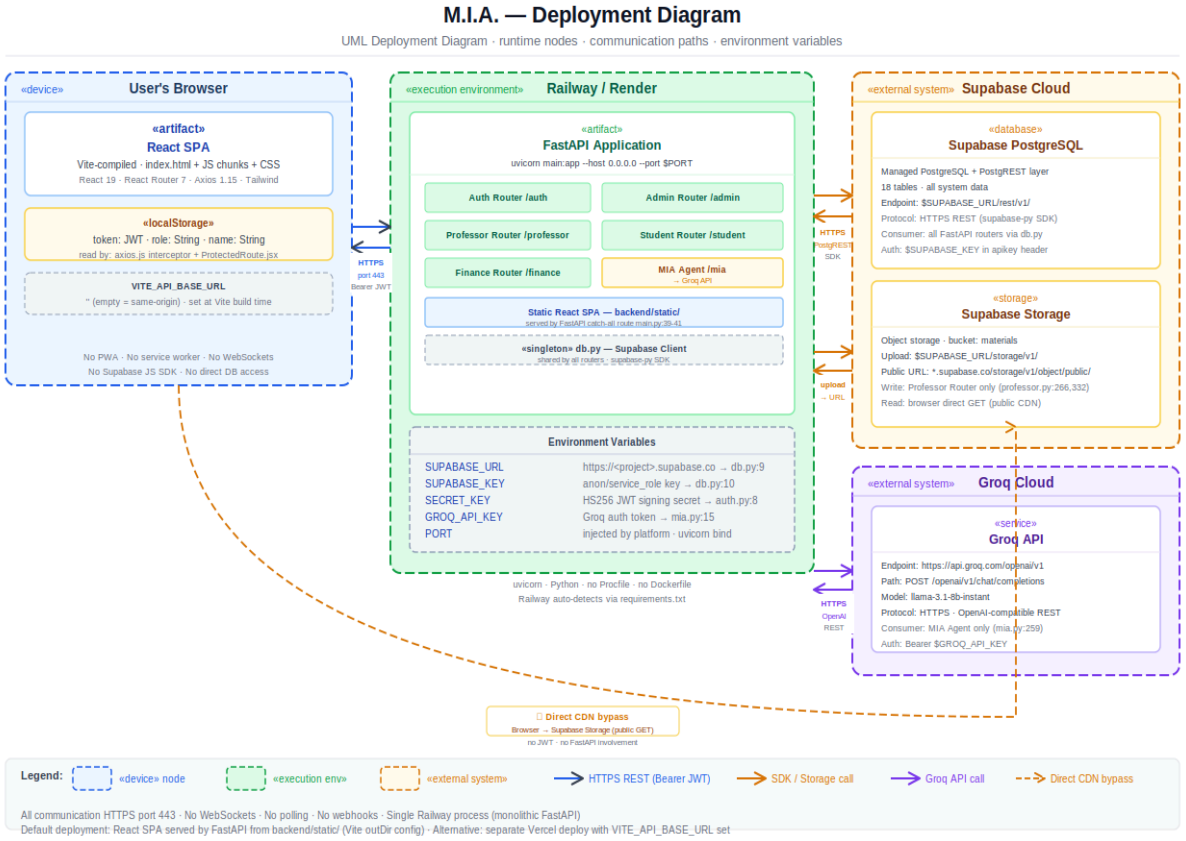

Figure 18: Deployment Diagram - runtime nodes, communication paths, and environment variable configuration

6.4 Technology Rationale

React was selected for the frontend because it provides a component-based architecture well-suited to building three distinct portals that share some UI patterns while diverging in functionality. Vite was chosen as the build tool for its superior developer experience and optimized production bundles. Tailwind CSS accelerates styling work by eliminating context-switching between HTML and CSS files.

FastAPI on the Python backend provides excellent performance in an async runtime environment, auto-generates OpenAPI documentation directly from code, and has strong typing support through Pydantic models. PostgreSQL is a reliable, ACID-compliant relational database well-suited to structured academic data with complex relationships. Supabase provides managed PostgreSQL hosting, eliminating operational overhead.

The Groq API provides sub-100 millisecond inference latency with high-quality language models, enabling the real-time response times necessary for a chat interface to feel responsive. All technologies were validated through working proof-of-concept implementations during the research phase. Groq latency was measured at under 200ms, well below the 500ms target.

7. Technical Implementation

7.1 Backend Implementation

The backend exposes RESTful endpoints following REST conventions. Authentication uses JWT tokens, enabling stateless operation that scales horizontally. All endpoints validate input using Pydantic models, catching malformed requests before they reach business logic. Responses are consistently formatted with meaningful error messages when validation fails.

Role-based access control is implemented at the endpoint level through direct calls to decode_token() from auth.py. Every protected endpoint verifies that the authenticated user has permission to access the requested resource: a student cannot access another student's grades, a professor cannot record grades for courses they do not teach, and administrator endpoints are inaccessible to students or professors. There is no middleware layer, as each router imports and calls decode_token() directly inside each endpoint handler.

The database is accessed exclusively through the Supabase Python SDK, which interfaces with the PostgREST API layer over Supabase PostgreSQL. The SDK singleton is instantiated once in db.py and shared across all routers. The finance module implements the most complex business logic: the create_major_fee() endpoint performs token validation, database insert, date calculation for installment scheduling, student lookup by major, and nested auto-creation of student_fee and installment records in a single request handler.

7.2 Frontend Implementation

The React frontend is organized into reusable components, each handling a specific UI concern. A Dashboard component displays key information. State management uses React hooks exclusively - useState manages local component state, and useEffect handles side effects like API calls. All API communication goes through a dedicated API client module (axios.js) with a JWT interceptor that reads the token from localStorage and injects it as a Bearer Authorization header on every outgoing request.

There is no React Context, no Redux, and no global state management library in the codebase. The auth state is stored exclusively in localStorage under three keys: token, role, and name. The Route Guard (ProtectedRoute.jsx) reads these keys to enforce role-based page access. Styling uses Tailwind CSS utility classes applied directly in component markup, with layouts that are responsive by default across mobile, tablet, and desktop viewports.

7.3 M.I.A AI Integration

The M.I.A. conversational adviser is implemented as a dedicated agent module (agent/mia.py), separate from the router files. When a student submits a chat message, the MIA Agent first validates the JWT, then assembles a comprehensive academic context by reading from six data stores: enrolled courses (via D5 enrollments + D4 courses), grades with GPA calculation (D6 grades), attendance statistics (D8 attendance), and student preferences (D14 student_preferences). The GPA calculation maps letter grades to numeric values with fallback handling and computes a weighted average. The attendance statistics compute session duration, presence percentage, and status distribution per course.

The assembled context, a system prompt defining the adviser role, the full conversation history from the messages table, and the current user message are combined into a request to the Groq API using the OpenAI Python SDK with a base URL override pointing to api.groq.com. The model used is llama-3.1-8b-instant. The call is blocking as the full response is generated server-side before FastAPI returns the HTTP response to the browser. Both the user message and the AI response are persisted in the messages table. The session_id is returned to the frontend with every response, enabling the student to resume conversations across separate browser sessions.

8. Features and Capabilities

8.1 Student Portal

Students access a detailed academic dashboard that provides an overview of their current semester. Enrolled courses appear with professor names and course materials. Current grades and GPA display prominently. Upcoming assignments show due dates. The Grades section displays grades by course and semester with GPA calculation, showing grade components and their weights to enable students to understand how their final grade is composed.

The Courses section shows full course details, including syllabus, professor information, and course materials organized by topic. Assignment management is centralized: students see all assignments with submission status, upload files or enter text, and view submission confirmation. The Transcript section provides a downloadable official academic transcript. The M.I.A. Chat feature provides AI-powered academic guidance in natural language with persistent conversation history and preference memory. The Tuition section shows financial information: current balance, payment schedule, paid and pending installments, and transaction history.

8.2 Professor Portal

The Professor Dashboard consolidates course management into one interface. My Courses lists all assigned courses with enrollment counts and quick access to course operations. The Grades section enables efficient grade recording with individual entry and weight validation or bulk upload via spreadsheet. Attendance tracking is organized by course and date, with the system calculating hours present and generating attendance reports. Assignments are created with title, description, due date, and optional file attachment. Course Materials are uploaded once and organized by topic. The Roster shows all enrolled students with quick access to individual grade and attendance summaries.

8.3 Administrator Portal

The Admin Dashboard provides an institutional overview with key metrics: total students and faculty, active courses, total enrollments, and recent activity. User Management enables the creation and editing of student and professor accounts with role assignment. System Reports provide data-driven institutional insights: enrollment reports by major and year, grade distribution across the institution, and payment collection status. Tuition Configuration enables administrators to define fees per major and academic program, with automatic assignment to students upon configuration. Payment tracking shows all transactions with student identification, amounts, and dates. The finance module implements automatic installment reconciliation: when a transaction is logged, the system marks installments paid in chronological order until the transaction amount is exhausted.

8.4 Phase 2 Enhancements

Features deferred to Phase 2 for future implementation include a scholarship management system with full UI, advanced financial analytics and reporting dashboards, mobile applications for iOS and Android, direct student-professor messaging, a grade appeal workflow, course evaluation surveys, WebSocket-based real-time notifications, and advanced institutional analytics. These features are well-understood architecturally but were not critical for MVP delivery within the June 2026 timeline.

9. Testing and Quality Assurance

9.1 Testing Strategy

The M.I.A. testing strategy follows a risk-based approach that prioritizes critical system paths: authentication and JWT validation, financial calculations (installment reconciliation, scholarship application, status computation), AI context assembly, and role-based access enforcement. The strategy is structured around four test levels applied progressively from unit isolation through full system integration, with specific targets for each level.

Test automation is central to the strategy. Unit and integration tests are executed on every pull request as a prerequisite for merge approval. End-to-end tests are run against a staging environment before each release. All tests are written before the code they cover (test-first approach) for business-critical functions in the finance module and MIA Agent.

9.2 Test Levels and Types

Unit Testing:

Unit tests isolate individual functions and validate their behavior in isolation. Key targets include: hash_password() and verify_password() in auth.py (correctness of bcrypt operations), _compute_status() in finance.py (boundary conditions: paid=0→Pending, paid=agreed→Settled, 0 < paid < agreed→Partial), _installment_dates() in finance.py (correct date calculation for Nov–Jun schedule), calculateGPA() in mia.py (grade mapping, edge cases: empty grades, all zeros), calculateAttendanceStats() (percentage calculation, status distribution), and validateTotalWeight() in GradeComponent (weight sum boundary at exactly 100%).

Integration Testing:

Integration tests validate that components work together correctly, focusing on router-to-database interactions. Key scenarios include the following: POST /auth/register creates records in both users and students/professors tables atomically; POST /admin/finance/major-fees creates the major_fee record, queries matching students, creates student_fee records, and auto-generates installments; DELETE /admin/finance/student-fees/{id} cascades correctly to installments and fee_transactions; POST /mia/chat with a valid JWT assembles context from all six data stores and returns a non-empty response.

System Testing:

System tests validate end-to-end workflows that span from the browser, through FastAPI, and into Supabase. Critical scenarios include the following: a full login flow for each role with correct dashboard redirection; professor grade recording with component weight validation rejection at 101%; student grade and attendance visibility after professor entry; complete payment lifecycle from major fee creation through student installment auto-generation through transaction logging and installment reconciliation; file upload through Supabase Storage with public URL retrieval and student access.

User Acceptance Testing (UAT):

UAT is conducted with representatives from each stakeholder group before production launch. Student UAT targets: can a student access their grades, find assignments, submit work, and get an M.I.A. response within 30 seconds of opening the portal? Professor UAT targets: can a professor record grades for all students in a course in under 5 minutes? Administrator UAT targets: can an admin create a new student account, enroll them in a course, configure a fee, and confirm installment generation without documentation?

10. Project Management

10.1 Team and Organization

The M.I.A. development team consists of five developers structured around specific domains. Xhoi Ikonomi leads frontend development and serves as project captain, responsible for overall timeline, team coordination, and React architecture decisions. Ajna Nushi focuses on student-facing frontend UI, including the M.I.A. chat interface and StudentDashboard component. Aida Bedaj implements core backend logic and database models, including the schema design and Pydantic validation layers. Parashqevi Klimi handles API routing, authentication, Supabase integration, and the finance module. Loric Hagard specializes in AI integration, prompt engineering, and LLM API optimization for the MIA Agent.

This structure provides clear responsibility boundaries while enabling collaboration. Frontend developers coordinate on component design and state management. Backend developers coordinate on API contracts and database schema. All developers participate in code review to maintain quality and knowledge sharing. The team meets weekly for status updates, feature demonstrations, blocker discussion, and sprint planning. Daily standups are asynchronous via Slack. Code is developed on feature branches, with a pull request review before merging into main.

10.2 Timeline and Milestones

The M.I.A. project spans three months from April through June 2026, structured into distinct phases. The research and validation phase runs through mid-April, where all technology choices are validated through proof-of-concept implementations. Design follows, establishing UI mockups and data models by late April. Three two-week development sprints proceed from late April through early June: Sprint 1 focuses on authentication and core API including the database schema and all Pydantic models; Sprint 2 covers academic features, including grades, attendance, assignments, and materials; Sprint 3 delivers the student portal and M.I.A. integration, including the Groq API connection and chat interface.

Testing and optimization occupies the second and third weeks of June. Quality assurance testing targets zero critical bugs before launch. Documentation is finalized during this period. Final deployment preparations occur in late June, with production deployment and user acceptance testing with representatives from each stakeholder group. The project culminates in late June with official go-live, followed by six months of post-launch support.

10.3 Risk Management

During planning, we identified major risks and established mitigation strategies. LLM API unavailability is mitigated by maintaining accounts with multiple providers (Groq and Gemini as fallback), implementing graceful degradation with helpful static responses when the LLM is unavailable, and monitoring API status. Database performance at scale is mitigated by load testing with 5,000+ simulated concurrent users during testing, query optimization before launch, and caching layer readiness. Scope creep is mitigated through strict scope enforcement, with all new requests deferred to Phase 2.

Comprehensive backup strategies, integrity validation tests, and a parallel-run period mitigate data migration errors. Security audits, code reviews that focus on the OWASP Top 10, and secure credential management via environment variables mitigate security vulnerabilities before launch. Poor M.I.A. accuracy is mitigated through beta testing with 50–100 real students, detailed feedback collection on answer quality, prompt refinement, and human adviser contact information provided as fallback.

11. Conclusion

M.I.A. represents a comprehensive solution to the fragmentation and manual processes that currently characterize university academic management at Metropolitan University of Tirana. By consolidating disparate systems into a unified platform with intelligent AI-powered guidance, the project directly addresses pain points affecting students, professors, and administrators on a daily basis.

The technology stack was carefully selected and validated through proof-of-concept work before development began. The five-layer architecture with its documented trade-offs, including the deliberate co-location of business logic and data access in flat router functions, the absence of a middleware layer in favor of direct decode_token() calls, and the single-process monolithic deployment model, reflects practical decisions suited to the project's scale and three-month timeline while remaining extensible for Phase 2.

The behavioral models documented in this report are; Activity Diagrams for Login, Grade Recording, and MIA Chat; State Diagrams for Assignment Lifecycle and Payment Status; and BPMN flows for Authentication, Assignment Submission, and the M.I.A. Conversational Flow.They provide a precise specification of system behavior that serves both as implementation guidance and as documentation for future maintainers.

The development team is structured for parallel work across frontend, backend, and AI components. Clear communication patterns and quality standards which are measured against the ISO/IEC 25010 quality model to ensure the system will be reliable and maintainable. Comprehensive risk management identifies potential obstacles and establishes mitigation strategies before they become problems.

Success is measured through specific, observable criteria: functional completeness across all three portals, performance targets including sub-200ms AI responses, user adoption rates of 80–90%, and business impact metrics demonstrating a 50% reduction in administrative manual effort. These criteria reflect both technical excellence and institutional value. M.I.A. is positioned to improve the academic experience significantly for all stakeholders at Metropolitan University of Tirana while reducing operational burden and enabling institutional scaling.